Alocação Dinâmica

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

- Considerando o estudo da **disciplina anterior**, tem-se a **definição do espaço alocado para uma variável** no momento de sua declaração.

- Isso significa que quando uma variável é **declarada**, tem-se que a **quantidade de memória** utilizada pela mesma é **conhecida**.

Introdução

- No caso de um vetor, é preciso saber inicialmente **quantos elementos podem ser armazenados no mesmo**.

- A **quantidade de elementos** que podem ser armazenados **influencia** na **quantidade de memória alocada** para o vetor.

Alocação Estática

- O **tipo de alocação** abordado na disciplina anterior é conhecido como **alocação estática de memória**.

- Neste contexto, não importa se a variável é **simples**, **vetor**, **matriz** ou **registro**, todas elas foram **declaradas** e tiveram o **espaço alocado** de **maneira estática**.

Observações

- A linguagem C permite alocar espaços de memória em **dois momentos distintos**:
  - Compilação
  - Execução

- Na **compilação** acontece a **alocação estática**. Na **execução** acontece a **alocação dinâmica**.

Alocação Dinâmica

- Por meio da **alocação dinâmica**, é possível **requisitar espaço** para **variáveis simples** e **compostas**, tais como **vetores**, **matrizes** e **registros**.

- Tal requisição acontece durante a **execução do programa**.

Alocação Dinâmica

- Existem funções na biblioteca **<stdlib.h>** relacionadas com a **alocação dinâmica**.

- Tais funções realizam a **alocação**, a **liberação** e a **realocação** de memória em **tempo de execução**.

Alocação Dinâmica de Memória

- A função básica para **alocar memória** é a **malloc**. Tal função recebe o **número de bytes que se deseja alocar** como **parâmetro**.

- Se a alocação for **bem sucedida**, o **endereço inicial da área de memória alocada é retornado**.

Alocação Dinâmica de Memória

- Se **não existir espaço livre suficiente para ser alocado**, a função malloc retorna um **endereço nulo**, que é representado pelo termo **NULL**.

- O termo **NULL** está definido em **<stdlib.h>**.

Exemplo

```
include<stdio.h>
include<stdlib.h>
int main(){
  // declaração de um ponteiro
  int *v;
  // exemplo de alocação dinâmica
  v = (int*) malloc (10*4);
  return 0;
}
```

Observações

- A função malloc retorna um **espaço de memória** do tipo **void**.
- Para definir um tipo para o espaço, realiza-se a **conversão de tipo**.
- Tal conversão acontece pela "**inserção**" do tipo do dado entre parênteses, **antes da chamada da função malloc**.

Observações

- Se a alocação for bem-sucedida, **v receberá um espaço de memória** suficiente para armazenar 40 bytes.

- Supondo que um valor do tipo inteiro utiliza **4 bytes**, esse espaço é **suficiente** para armazenar **10 valores inteiros**.

Observações

- Se a alocação **não for** bem sucedida, **NULL** é retornado.

- É possível **checar o retorno** da função malloc. Dependendo do valor de retorno, é possível saber o **resultado da alocação**.

Exemplo

```
...
v = (int*) malloc(10*sizeof(int));
if (v==NULL)
{
   printf("Memoria insuficiente.\n");
   exit(1);  /* aborta o programa e retorna 1 para o sist. operacional */
}
...
```

Liberação Dinâmica de Memória

- Para **liberar um espaço de memória** alocado de **forma dinâmica**, utiliza-se a função **free**.

- Essa função recebe o **ponteiro** que referencia o **espaço de memória** a ser **liberado** como **parâmetro**.

Exemplo

```
include<stdio.h>
include<stdlib.h>
int main(){
  int *v;
  v = (int*) malloc (10*4);
  free(v);
  return 0;
}
```

Referências

- Introdução a Estruturas de Dados – 1ª edição. Waldemar Celes, Renato Cerqueira e José Lucas Rangel. Campus, 2004.

- Aula Professor Wesley Romão UEM.